

Apache Kafka & KRaft

The Ultimate Interview & Production Guide

1. Kafka brokers: servers that store and serve partitions and handle producer/consumer traffic.
2. Topics and partitions: logical streams broken into ordered logs for parallelism and scalability.
3. Producers: push records to leader partition, use keys, batching, and compression for throughput.
4. Consumers: read from offsets and work in consumer groups to parallelize work safely.
5. Consumer offsets: committed to `__consumer_offsets` to support restart and replay.
6. ZooKeeper: older Kafka coordination layer for metadata, brokers, and controller election.
7. KRaft: Kafka-native Raft metadata quorum without external ZooKeeper dependency.
8. ISR: In-Sync Replicas that are fully caught up and eligible to become partition leaders.
9. Data durability: replication factor, `acks=all`, and `min.insync.replicas` protect against loss.

Interview Questions and Scenarios

Q: What is Kafka? A: Distributed event streaming platform for high-throughput, durable pipelines.

Q: What is a consumer group? A: A set of consumers that share partition ownership.

Q: What is an offset? A: The position of a message within a partition.

Q: Why use Kafka? A: Decoupling, replayability, scalability, fault tolerance, and throughput.

Scenario: Consumer lag rising rapidly. Action: scale consumers, add partitions, and monitor lag.

Scenario: Broker fails. Action: use replication and ISR elections to maintain availability.

Scenario: Data loss during writes. Action: verify `acks=all`, replication factor, and `min.insync.replicas`.

Scenario: Duplicate processing. Action: use idempotent producers and dedupe logic in consumers.

Scenario: Consumer rebalance loops. Action: tune `session.timeout.ms` and `max.poll.interval.ms`.

Scenario: Payment processing requires exactly-once semantics. Use Kafka transactions + `read_committed`.

Production answer: Kafka provides at-least-once by default; exactly-once requires deliberate design.

Production Configs and Monitoring

Producer config: bootstrap.servers, acks=all, enable.idempotence=true, linger.ms=20, compression.type=zstd.

Consumer config: group.id, auto.offset.reset=earliest, enable.auto.commit=false, max.poll.records=500.

Topic config: num.partitions, replication.factor, min.insync.replicas, retention.ms, cleanup.policy.

Do: use 3x replication, monitor lag, disk, and request latency in production.

Dont: run single-broker production, ignore under-replicated partitions, or set acks=0 for critical topics.

Metrics to monitor: underreplicated partitions, offline partitions, request queue time, lag, and GC pauses.

Prometheus: scrape Kafka JMX exporter on port 7071 and visualize with Grafana.

Example topic command: kafka-topics.sh --bootstrap-server localhost:9092 --create --topic prod.orders --partitions 6 --replication

Example describe command: kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group payment-process

Final summary: Kafka is built for durable, high-throughput, decoupled event streaming with modern KRaft leadership.